

CSE421: Computer Networks

Chapter 4: The Transport Layer — TCP Part III (Congestion Control)

Exam Study Guide

July 15, 2026

Contents

1	Introduction to Congestion Control	2
1.1	Congestion Control vs. Flow Control	2
1.2	When Does Congestion Happen?	2
1.3	The Congestion Collapse Feedback Loop	2
2	Congestion Control Methods	2
2.1	Open-Loop Congestion Control (Prevention)	3
2.2	Closed-Loop Congestion Control (Remediation)	3
3	The TCP Window State Variables	3
4	TCP Congestion Control Algorithm Phases	4
4.1	1. Slow Start Phase (Exponential Growth)	4
4.1.1	ACK-Based Growth Example	4
4.2	2. Congestion Avoidance Phase (Linear Growth)	4
4.3	3. Handling Packet Loss & State Transitions	4
4.3.1	Scenario A: Retransmission Timeout (RTO) — Severe Congestion	5
4.3.2	Scenario B: Three Duplicate ACKs — Mild Congestion (Fast Recovery)	5
5	Congestion Control Decision Flowchart	5
6	Detailed Mathematical Study Example	7
6.1	Step-by-Step Round Log	7
6.2	Triggering Loss Events	7

1 Introduction to Congestion Control

In any packet-switched network, multiple hosts compete for shared physical resources (such as router buffers, processing capacity, and transmission links). When too much data is sent across the network, performance degrades significantly. This phenomenon is known as **Congestion**.

1.1 Congestion Control vs. Flow Control

It is crucial to distinguish between these two mechanisms for the exam:

- **Flow Control:** A point-to-point mechanism where the **Receiver** limits the sender's speed to prevent its local application buffer from overflowing.
- **Congestion Control:** A network-wide mechanism controlled primarily by the **Sender** to prevent the physical intermediate routers and transmission links from becoming overwhelmed.

1.2 When Does Congestion Happen?

Congestion is primarily triggered by two main bottlenecks:

1. Capacity Exhaustion:

$$\text{Network Load} \gg \text{Network Capacity}$$

The combined transmission rates of all active senders exceed the physical bandwidth of the links.

2. Router Buffer Bottlenecks: Routers process incoming packets using physical memory queues (input and output buffers).

- **Packet Arrival Rate > Router Processing Rate:** Packets accumulate in the input buffer queue.
- **Packet Departure Rate < Router Processing Rate:** Packets cannot leave the output buffer fast enough to match the routing speed, causing output queues to overflow and drop packets.

1.3 The Congestion Collapse Feedback Loop

When packets are delayed due to long router queues, sender timers expire (Retransmission Timeout - RTO). Because the sender has not received an ACK, it assumes the packet was lost and transmits it again. This introduces even more redundant packets into the already clogged network, reducing throughput to near zero.

2 Congestion Control Methods

Congestion control techniques are broadly divided into two strategies: **Open-Loop** (prevention before congestion occurs) and **Closed-Loop** (remediation after congestion is detected).

2.1 Open-Loop Congestion Control (Prevention)

These policies are implemented during protocol design to prevent congestion from occurring in the first place:

- **Selective Repeat Protocol:** Avoids unnecessary retransmissions by only sending back lost packets, preventing wasted link capacity.
- **Discarding Policy:** Routers discard less critical packets (such as audio/video streaming packets) to prioritize critical system control traffic when queues begin to fill.
- **Admission Policy:** In virtual-circuit networks, routers inspect path capacities before accepting a new connection request. If resources are scarce, the network denies connection setup.

2.2 Closed-Loop Congestion Control (Remediation)

These mechanisms dynamically monitor the active network state to correct congestion:

- **Backpressure (Hop-by-Hop):** When a node becomes congested, it stops receiving packets from its direct upstream neighbor. This neighbor's buffer then fills up, forcing it to stop receiving from its upstream neighbor. This pressure propagates step-by-step backward until it reaches the source.
- **Choke Packet:** The congested router generates a dedicated control packet (called a choke packet) and sends it directly to the source host, commanding it to reduce its transmission rate.
- **Implicit Signaling:** The sender infers congestion without explicit messages from the network. It observes long packet delays, high packet loss, or a lack of ACKs, and automatically slows down.
- **Explicit Signaling:** Congested routers mark specific control bits inside transport headers (such as ECN - Explicit Congestion Notification) of existing packets to explicitly notify either the destination (Forward Signaling) or the source (Backward Signaling) of the congestion.

—

3 The TCP Window State Variables

The volume of unacknowledged bytes a sender can transmit is restricted by two concurrent variables:

- **rwnd (Receiver Window):** The free buffer space remaining on the receiving host (enforces flow control).
- **cwnd (Congestion Window):** The maximum amount of unacknowledged data the sender estimates the network can carry safely (enforces congestion control).

The actual allowable size of the sender's current transmission window is defined by the minimum of these two variables:

$$\text{Actual Send Window} = \min(\text{rwnd}, \text{cwnd})$$

—

4 TCP Congestion Control Algorithm Phases

Modern TCP dynamically modifies the size of *cwnd* through sequential algorithms: **Slow Start**, **Congestion Avoidance**, and **Fast Recovery**.

4.1 1. Slow Start Phase (Exponential Growth)

In this phase, the sender does not know the network's capacity. It starts with a conservative initial congestion window size of **1 MSS** (Maximum Segment Size) and aggressively probes the network.

The Slow Start Growth Rule

For **every single individual ACK** received, *cwnd* increases by 1 MSS:

$$\text{New } cwnd = cwnd + 1 \text{ MSS}$$

When looking at round-trip times (RTTs), this results in the window doubling each round:

$$cwnd = 2^n \text{ MSS (where } n \text{ is the RTT round)}$$

4.1.1 ACK-Based Growth Example

Assume a sender currently has *cwnd* = 4 MSS, meaning it has sent 4 outstanding segments (Segments 1, 2, 3, and 4):

- The client receives the ACK for Segment 1. It increases *cwnd* by 1 MSS (now *cwnd* = 5 MSS). Since Segment 1 is ACKed, only 3 segments are outstanding. The sender can immediately transmit 2 new segments (representing the ACKed slot + the newly opened window space).
- As the ACKs for Segment 2, 3, and 4 arrive, each ACK increments *cwnd* by 1 MSS.
- Once all 4 ACKs are processed, the total window size increases to $4 + 4 = 8$ MSS. The window has successfully doubled.

—

4.2 2. Congestion Avoidance Phase (Linear Growth)

The exponential growth of the Slow Start phase cannot continue indefinitely. TCP uses a threshold parameter called **ssthresh** (Slow Start Threshold) to transition safely.

Once *cwnd* ≥ *ssthresh*, the sender switches to the **Congestion Avoidance Phase**. In this state, *cwnd* grows linearly, increasing by exactly 1 MSS for every full round-trip time (RTT):

$$\text{For each received ACK: New } cwnd = cwnd + \frac{1}{cwnd} \text{ MSS}$$

—

4.3 3. Handling Packet Loss & State Transitions

When a packet is lost, it signals congestion. TCP detects loss via two distinct events, handling them differently:

4.3.1 Scenario A: Retransmission Timeout (RTO) — Severe Congestion

When the retransmission timer expires without receiving an ACK:

1. Set the new threshold to half of the current congestion window:

$$ssthresh = \max \left(\left\lfloor \frac{cwnd}{2} \right\rfloor, 2 \text{ MSS} \right)$$

2. Reset *cwnd* to 1 MSS:

$$cwnd = 1 \text{ MSS}$$

3. Re-enter the **Slow Start Phase** to slowly rebuild throughput.

4.3.2 Scenario B: Three Duplicate ACKs — Mild Congestion (Fast Recovery)

When the sender receives 3 duplicate ACKs (4 identical ACKs total), it knows the packet was likely dropped due to temporary buffer congestion, but subsequent packets are still getting through.

1. Adjust the threshold:

$$ssthresh = \max \left(\left\lfloor \frac{cwnd}{2} \right\rfloor, 2 \text{ MSS} \right)$$

2. Set the new *cwnd* to the threshold level (in TCP Reno, this is *ssthresh* or *ssthresh* + 3 MSS to account for buffered segments):

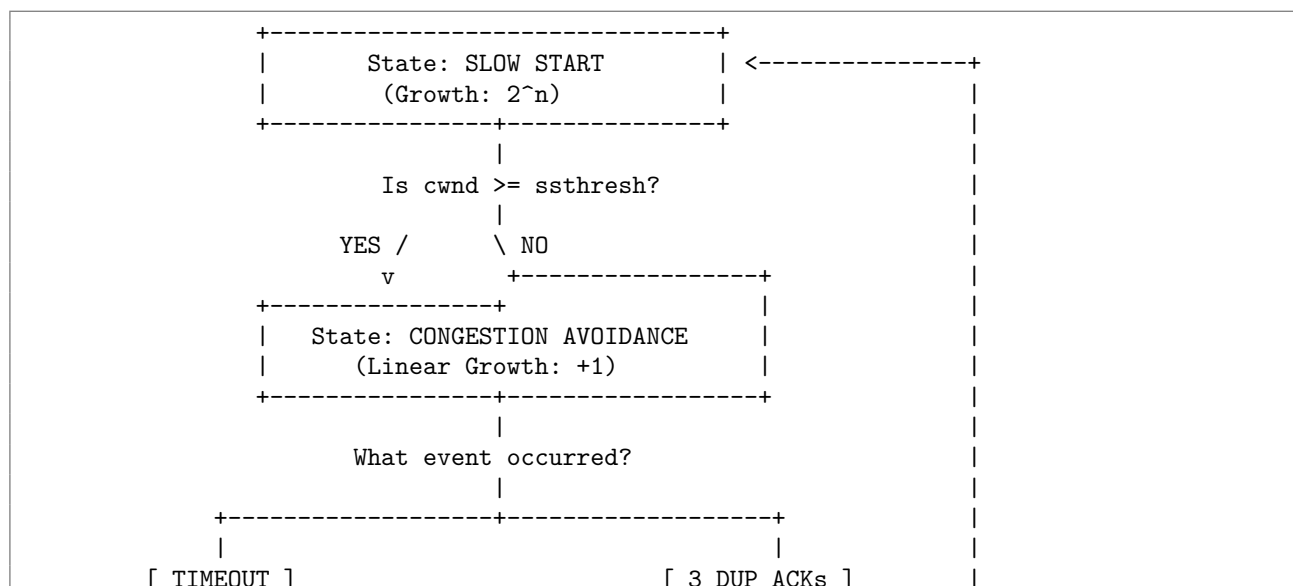
$$cwnd = ssthresh$$

3. Skip Slow Start entirely and enter the **Congestion Avoidance Phase** directly (linear increase).

—

5 Congestion Control Decision Flowchart

The decision logic of the TCP congestion control engine is visualized below:



|
ssthresh = cwnd / 2
cwnd = 1 MSS
|
v
Go to SLOW START

| |
ssthresh = cwnd / 2 |
cwnd = ssthresh |
| |
v |
Go to CONG. AVOIDANCE --+

6 Detailed Mathematical Study Example

Scenario: A TCP connection is established with an initial $ssthresh = 16$ MSS. The sender transmits segments in successive RTT rounds. Assume a maximum capacity limit where loss is triggered.

6.1 Step-by-Step Round Log

1. **Round 1:** $cwnd = 1$ MSS. (State: Slow Start, $cwnd < ssthresh$).
2. **Round 2:** $cwnd = 2$ MSS. (Slow Start).
3. **Round 3:** $cwnd = 4$ MSS. (Slow Start).
4. **Round 4:** $cwnd = 8$ MSS. (Slow Start).
5. **Round 5:** $cwnd = 16$ MSS. (Slow Start).
 - **Note:** Since $cwnd$ has reached $ssthresh$ ($16 \geq 16$), the protocol transitions to **Congestion Avoidance** for the next round.
6. **Round 6:** $cwnd = 17$ MSS. (Congestion Avoidance, linear +1).
7. **Round 7:** $cwnd = 18$ MSS. (Congestion Avoidance).
8. **Round 8:** $cwnd = 19$ MSS. (Congestion Avoidance).
9. **Round 9:** $cwnd = 20$ MSS. (Congestion Avoidance).

6.2 Triggering Loss Events

Suppose at the end of Round 9 (while $cwnd = 20$ MSS), we trace two alternative scenarios for our exam preparation:

Scenario A: Timeout Occurs at $cwnd$

1. Calculate new threshold:

$$ssthresh = \max\left(\frac{20}{2}, 2\right) = 10 \text{ MSS}$$

2. Reset window:

$$cwnd = 1 \text{ MSS}$$

3. **Next Rounds (Slow Start up to 10, then Linear):**

- Round 10: $cwnd = 1$ MSS (Slow Start)
- Round 11: $cwnd = 2$ MSS (Slow Start)
- Round 12: $cwnd = 4$ MSS (Slow Start)
- Round 13: $cwnd = 8$ MSS (Slow Start)
- Round 14: $cwnd = 10$ MSS (Slow Start. Threshold reached!)
- Round 15: $cwnd = 11$ MSS (Congestion Avoidance, linear +1)

Scenario B: 3 Duplicate ACKs Arrive at cwnd

1. Calculate new threshold:

$$ssthresh = \max\left(\frac{20}{2}, 2\right) = 10 \text{ MSS}$$

2. Set new window directly to threshold:

$$cwnd = 10 \text{ MSS}$$

3. Next Rounds (Directly in Congestion Avoidance):

- Round 10: $cwnd = 10 \text{ MSS}$ (Congestion Avoidance, linear growth)
- Round 11: $cwnd = 11 \text{ MSS}$ (Congestion Avoidance, +1)
- Round 12: $cwnd = 12 \text{ MSS}$ (Congestion Avoidance, +1)